

Trabajo práctico 4

Sección 1

Introducción a Redis

Inciso 1

¿Qué tipo de base de datos es Redis? ¿En qué se diferencia de una base de datos relacional y de otras bases de datos NoSQL como MongoDB?

Respuesta:

Redis es una base de datos **NoSQL** perteneciente a la familia de los almacenes de **clave-valor**. A menudo se le denomina un **servidor de estructura de datos**, ya que a diferencia de otros almacenes de clave-valor puros, permite que los valores no sean simples "blobs" opacos, sino estructuras más complejas.

Diferencias con una base de datos relacional (RDBMS)

- **Modelo de datos:** Mientras que una RDBMS organiza la información en **tablas con filas y columnas** rígidas, Redis utiliza un modelo **orientado a agregados** donde los datos relacionados se almacenan juntos bajo una clave única.
- **Esquema:** Las bases de datos relacionales requieren un **esquema fijo** definido de antemano. Redis es **"sin esquema" (schemaless)**, lo que permite guardar cualquier dato bajo una clave sin restricciones estructurales previas.
- **Acceso a datos:** En una RDBMS se utilizan consultas **SQL** complejas que pueden involucrar uniones (*joins*) de múltiples tablas. En Redis, el acceso se realiza primordialmente a través de la **clave principal**, lo que ofrece un rendimiento superior para operaciones simples.
- **Escalabilidad:** Las RDBMS están diseñadas típicamente para el escalado vertical, mientras que Redis y otros sistemas

NoSQL facilitan el **escalado horizontal** mediante la ejecución en clústeres.

Diferencias con otras NoSQL (como MongoDB)

Aunque ambas son orientadas a agregados y operan sin esquema, existen distinciones técnicas importantes:

- **Naturaleza del valor:** MongoDB es una **base de datos de documentos**. Esto significa que los valores (documentos JSON/BSO) son **examinables** por la base de datos. Redis, como almacén de clave-valor, tiende a tratar el valor como un **bloque opaco** para el motor, aunque ofrece estructuras internas específicas como listas, conjuntos y hashes.
- **Capacidad de consulta:** En MongoDB se pueden realizar **consultas basadas en los campos internos** del documento, crear índices sobre esos campos y recuperar solo partes del agregado. En Redis, generalmente se espera recuperar el agregado completo basándose únicamente en su **clave**.
- **Funcionalidad extendida:** Redis destaca por permitir operaciones de **rango, unión e intersección** directamente sobre las estructuras de datos que almacena (como listas o conjuntos), algo que no es la función primaria de un almacén de documentos como MongoDB.

Inciso 2

¿Dónde almacena los datos Redis? ¿Qué implicancias tiene esto en términos de velocidad y de persistencia?

Respuesta:

Redis almacena sus datos primordialmente en la **memoria principal (RAM)**.

Este método de almacenamiento tiene implicancias fundamentales en dos aspectos:

1. Velocidad y Rendimiento

- **Baja latencia:** Al residir los datos en RAM, se obtiene una **ventaja de rendimiento masiva**, ya que el motor no tiene que gestionar la lenta entrada/salida (E/S) del disco al procesar cada evento o consulta.
- **Alta capacidad de respuesta:** Esto permite que Redis ofrezca una **capacidad de respuesta considerablemente mayor** a las solicitudes en comparación con los sistemas que dependen de la persistencia física inmediata.

2. Persistencia y Durabilidad

- **Durabilidad relajada:** Redis sacrifica la durabilidad estricta en favor del rendimiento. Aplica las actualizaciones en su representación en memoria y luego **vacía periódicamente los cambios al disco** (respaldo).
- **Riesgo de pérdida de datos:** Si el servidor falla, se **perderán las actualizaciones** realizadas desde el último vaciado al disco. Por esta razón, se considera ideal para datos cuya pérdida no sea crítica o "trágica", como la gestión de **sesiones de usuario** o carritos de compra transitorios.
- **Limitación de volumen:** Una restricción técnica obvia es que el conjunto de datos de trabajo debe poder **cabere completamente en la memoria RAM** disponible del servidor.

Inciso 3

¿Qué tipos de datos soporta Redis? Listar y describir brevemente cada uno.

Respuesta

- **Strings (Cadenas):** El tipo de dato más básico. Pueden contener cualquier tipo de datos binarios, como texto, imágenes JPEG serializadas o JSON. Tienen un límite

máximo de 512 MB. También soportan operaciones numéricas (incrementar/decrementar).

- **Lists (Listas):** Colecciones de cadenas de texto ordenadas por el orden de inserción. Básicamente son listas doblemente enlazadas, lo que permite añadir elementos al inicio (**L****P****U****S****H**) o al final (**R****P****U****S****H**) en tiempo constante $O(1)$. Ideal para colas de mensajes.
- **Sets (Conjuntos):** Colecciones no ordenadas de cadenas de texto únicas. Permiten hacer operaciones de conjuntos muy rápidas como intersecciones (**S****I****N****T****E****R**), uniones (**S****U****N****I****O****N**) y diferencias.
- **Sorted Sets / Zsets (Conjuntos Ordenados):** Similares a los Sets ordinarios, pero cada elemento está asociado a un puntaje numérico (*score*). Los elementos se mantienen ordenados automáticamente por este puntaje. Son perfectos para tablas de clasificación (*leaderboards*).
- **Hashes:** Mapas de campos y valores (básicamente objetos JSON planos o diccionarios). Son ideales para representar objetos del mundo real (por ejemplo, un usuario con campos como **n****a****m****e**, **a****g****e**, **e****m****a****i****l**).
- **Bitmaps (Mapas de bits):** No son un tipo de dato independiente, sino operaciones especiales sobre las *Strings* que te permiten tratarlas como un vector de bits. Muy eficientes para análisis de comportamiento y contadores de alta densidad.
- **Bitfields (Campos de bits):** Permiten codificar múltiples campos de números enteros de diferentes longitudes de bits dentro de un string de Redis.
- **HyperLogLogs:** Una estructura de datos probabilística utilizada para estimar la cardinalidad (el número de elementos únicos) de un conjunto con un uso de memoria extremadamente bajo (un máximo de 12 KB por clave), con un margen de error inferior al 1%.
- **Geospatial (Geoespacial):** Te permite almacenar coordenadas de latitud y longitud y buscar elementos

cercanos en un radio específico. Internamente, utiliza *Sorted Sets* y *Geohashes*.

- **Streams:** Una estructura de datos que actúa como un registro de datos solo de anexión (*append-only log*). Diseñada específicamente para arquitecturas de eventos, mensajería en tiempo real y soporte para grupos de consumidores (similar conceptualmente a Kafka).

Inciso 4

Enunciar las características principales de Redis

Respuesta

- **Modelo de Datos Orientado a Agregados:** Almacena datos relacionados juntos bajo una clave única, lo que facilita la interacción con el almacenamiento en términos de unidades completas.
- **Almacenamiento en Memoria:** Almacena los datos primordialmente en la **memoria principal (RAM)**.
- **Estructuras de Datos Enriquecidas:** No trata los valores como simples bloques opacos (blobs); permite descomponer agregados en estructuras específicas como **listas, conjuntos (sets) y hashes**.
- **Operaciones Avanzadas:** Permite realizar operaciones complejas directamente en el servidor, tales como operaciones de **rango** sobre listas, o **unión, intersección y diferencia** sobre conjuntos.
- **Escalabilidad Horizontal:** Está diseñado para ejecutarse eficientemente en **clústeres**, lo que permite distribuir la carga entre múltiples nodos y gestionar grandes volúmenes de datos mediante fragmentación (*sharding*).
- **Persistencia y Durabilidad Relajada:** Prioriza el rendimiento aplicando actualizaciones en memoria y volcándolas periódicamente al disco.
- **Simplicidad de Acceso:** El acceso a los datos se realiza primordialmente a través de la **clave principal**, lo que

simplifica la API y optimiza la velocidad de recuperación de información.

Inciso 5

Comparar Redis con los RDBMS: ¿en qué casos conviene usar Redis en lugar de una base de datos relacional y en cuáles no.

Respuesta:

¿Cuándo conviene usar Redis en lugar de un RDBMS?

1. **Necesidad de alto rendimiento y baja latencia**
2. **Gestión de datos transitorios o "no críticos":** Es excelente para información cuya pérdida parcial ante un fallo no sea una "tragedia", como el **estado de sesión del usuario** o **carritos de compra** temporales.
3. **Acceso simple mediante clave primaria**
4. **Operaciones sobre estructuras de datos específicas:** Cuando se requiere realizar operaciones de **rango** sobre listas, o **unión e intersección** sobre conjuntos directamente en el motor de base de datos, Redis es superior debido a que entiende estas estructuras de forma nativa.
5. **Escalabilidad horizontal:** Redis está diseñado para ejecutarse más fácilmente en clústeres

¿Cuándo NO conviene usar Redis (y preferir un RDBMS)?

1. **Relaciones complejas entre datos:** Si la aplicación necesita correlacionar diferentes conjuntos de información o navegar por vínculos profundos, los almacenes de clave-valor no son la mejor solución.
2. **Transacciones multi-operación:** Si se requiere asegurar que múltiples cambios en distintas claves se realicen como una única unidad atómica (propiedades **ACID** puras), el RDBMS es la opción correcta.
3. **Consultas basadas en el contenido**
4. **Garantía total de persistencia y durabilidad**

5. Volumen de datos superior a la RAM

Inciso 6

¿Redis tiene soporte para transacciones? ¿Cómo funcionan?
¿Qué garantías ofrecen y qué limitaciones tienen respecto de las transacciones ACID?

Respuesta

Redis, al igual que otras bases de datos orientadas a agregados, posee un soporte para transacciones que difiere significativamente del modelo relacional tradicional, enfocándose principalmente en la unidad del **agregado**

1. ¿Cómo funcionan las transacciones en Redis?

Las transacciones en Redis se basan en el concepto de **orientación a agregados**. A diferencia de los RDBMS que permiten manipular cualquier combinación de filas de diferentes tablas, Redis trata el **agregado individual** (el valor asociado a una clave, que puede ser una lista, conjunto o hash) como la unidad mínima de interacción.

- **Unidad Atómica:** Las operaciones son atómicas únicamente dentro de los límites de un **único agregado**.
- **Estructuras de Datos:** Redis permite que los valores no sean opacos, sino que el motor comprenda estructuras como listas o hashes, permitiendo realizar operaciones complejas (como uniones o rangos) directamente sobre ellos de forma atómica.

2. Garantías ofrecidas

Redis ofrece garantías transaccionales útiles pero acotadas al nivel de acceso por clave:

- **Atomicidad a nivel de Agregado:** El sistema garantiza que la manipulación de un solo agregado a la vez sea atómica.

- **Consistencia Lógica:** Asegura que los elementos dentro de un mismo agregado tengan sentido mutuo (consistencia interna).
- **Aislamiento:** Al operar sobre agregados, la base de datos garantiza que los datos relacionados residan en el mismo nodo, facilitando el control de concurrencia sobre esa unidad.

3. Limitaciones respecto de las transacciones ACID

Aunque Redis proporciona mecanismos transaccionales, presenta limitaciones claras frente a las propiedades **ACID** (Atómicas, Consistentes, Aisladas y Duraderas) de los sistemas relacionales:

- **Alcance Multi-Agregado (Atomicidad):** Generalmente, Redis no admite transacciones ACID que abarquen múltiples agregados (múltiples claves independientes).
- **Falta de Rollback Automático:** Si se intentan guardar varias claves y una falla, el sistema no suele revertir o deshacer automáticamente las operaciones previas exitosas del conjunto.
- **Durabilidad Relajada:** A diferencia de la durabilidad estricta de un RDBMS (enfocada en disco), Redis prioriza el rendimiento almacenando datos en **RAM** y volcándolos periódicamente al disco. Esto implica que, ante un fallo del servidor, se pueden **perder las actualizaciones** realizadas desde el último vaciado.
- **Consistencia entre Agregados:** Si bien se mantiene la coherencia dentro de un agregado, no se garantiza la consistencia lógica entre diferentes agregados fuera del marco de la aplicación.

1: **Agregado** = Un conjunto de objetos que:

- Están relacionados entre sí
- Se guardan y recuperan **juntos**
- Se modifican **juntos** (atómicamente)

- Tienen un **límite claro**

Inciso 7

¿Redis tiene persistencia? Describir los mecanismos disponibles (RDB y AOF) e indicar las diferencias entre ellos

Respuesta

Sí, Redis sí tiene persistencia.

Redis maneja principalmente dos mecanismos de persistencia: **RDB (Redis Database)** y **AOF (Append Only File)**. Puedes usar uno, ambos, o incluso ninguno (si solo quieres usar Redis como caché volátil).

1. RDB (Redis Database Snapshotting)

El mecanismo RDB realiza **instantáneas (snapshots)** de tus datos en momentos específicos del tiempo y los guarda como un archivo binario compacto (generalmente llamado `dump.rdb`).

- **¿Cómo funciona?** Configuras reglas basadas en tiempo y cantidad de cambios (por ejemplo: *"si cambian al menos 1000 claves en 60 segundos, haz un snapshot"*). Redis hace un `fork()` de su proceso; el proceso hijo escribe los datos en el archivo RDB de forma asíncrona mientras el proceso principal sigue atendiendo peticiones.

2. AOF (Append Only File)

El mecanismo AOF registra **cada operación de escritura** que recibe el servidor en un archivo de texto (generalmente `appendonly.aof`). Es un historial de comandos.

- **¿Cómo funciona?** Cada vez que ejecutas un comando que modifica datos (como `SET` o `HSET`), Redis lo anota al final del archivo. Tiene diferentes políticas para asegurar

los datos en el disco (**fsync**): cada segundo (por defecto), con cada comando, o dejarlo en manos del sistema operativo.

- **¿Qué es el AOF Rewrite?** Como el archivo puede crecer demasiado, Redis incluye un mecanismo automático para "reescribir" el archivo en segundo plano, creando la versión más corta posible de comandos necesarios para reconstruir el estado actual de los datos.

3. Comparativa

Característica	RDB (Snapshotting)	AOF (Append Only File)
Estrategia	Instantáneas en puntos del tiempo.	Registro continuo de comandos de escritura.
Tamaño del archivo	Pequeño y compacto (binario).	Grande (texto con el historial de comandos).
Pérdida de datos	Mayor (desde el último snapshot).	Mínima (máximo 1 segundo por defecto).
Impacto en Rendimiento	Mínimo (el proceso hijo hace el trabajo).	Ligero impacto constante por escritura en disco.
Velocidad de Recuperación	Muy rápida.	Más lenta (tiene que recrear comando por comando).

Inciso 8

¿Cuáles son los principales casos de uso de Redis en aplicaciones reales?

Respuesta

1. Almacenamiento en Caché (Caching)

- Es el caso de uso más común. Para evitar saturar la base de datos principal con consultas repetitivas, se guardan los resultados en Redis. Como Redis responde en microsegundos, la aplicación vuela.
2. Gestión de Sesiones (Session Store)
 - En aplicaciones web distribuidas (que corren en varios servidores en la nube), si un usuario inicia sesión en el Servidor A y su siguiente click cae en el Servidor B, el Servidor B necesita saber quién es.
 3. Limitación de Tasa (Rate Limiting)
 - Para proteger una API de abusos, ataques de fuerza bruta o raspado de datos (scraping), necesitás contar cuántas peticiones hace un usuario en un período de tiempo. Hacer esto en una base de datos común destruiría el rendimiento del disco.
 4. Tablas de Clasificación en Tiempo Real (Leaderboards)
 - Manejar un ranking de millones de usuarios en tiempo real (como en videojuegos o plataformas de trading) usando **ORDER BY** en SQL es una pesadilla de rendimiento.
 5. Colas de Mensajes y Sistemas Pub/Sub
 - Redis permite la comunicación en tiempo real entre diferentes microservicios de tu arquitectura.
 6. Almacenamiento de Datos Geoespaciales (Geospatial)
 - Redis incluye comandos específicos para manejar coordenadas de longitud y latitud, permitiendo calcular distancias o buscar elementos cercanos.

Sección 2

Manejo de sesiones

I. Valores de texto

Inciso 9

Agregar una clave package con el valor "Bariloche 3 days".

```
SET package "Bariloche 3 days"
```

Inciso 10

Agregar una clave user con el valor "Turismo BD2". Obtener el valor de la clave user.

```
SET user "Turismo BD2"  
GET user
```

Inciso 11

Obtener todas las claves almacenadas actualmente.

```
KEYS *
```

Inciso 12

Agregar una clave user con el valor "Cronos Turismo". ¿Cuál es el valor actual de la clave user?

```
SET user "Cronos Turismo"  
GET user
```

Tiene cronos turismo

Inciso 13

Concatenar " S.A." a la clave user. ¿Cuál es el valor actual de la clave user?

```
APPEND user " S.A."  
GET user
```

"Cronos Turismo S.A." es su valor.

Inciso 14

Eliminar la clave user. ¿Qué valor retorna si se intenta obtener la clave user luego de eliminarla?

```
DEL user  
GET user
```

El valor de user es nil

II. Valores de numéricos

Inciso 15

Verificar si existe la clave visits.

```
EXISTS visits
```

Visits no existe

Inciso 16

Agregar una clave visits con el valor 0.

```
SET visits 0
```

Inciso 17

Incrementar en 1 la clave visits. ¿Cuál es el valor actual?

```
INCR visits
```

Tiene el valor 1

Inciso 18

Incrementar en 5 la clave visits. ¿Cuál es el valor actual?

```
INCRBY visits 5
```

Posee el valor 6

Inciso 19

Decrementar en 1 la clave visits. ¿Cuál es el valor actual?

```
DECR visits  
(integer) 5
```

Inciso 20

Incrementar en 2 la clave visits. ¿Cuál es el valor actual?

```
DECRBY visits 2
```

El valor es 3

Inciso 21

Agregar una clave "value package" con el valor 539789.32.

```
SET "value package" 539789.32  
GET "value package"
```

Resultado: "539789.32"

Inciso 22

Incrementar en 20000 la clave "value package". ¿Cuál es el valor actual?

```
INCRBYFLOAT "value package" 20000
```

Inciso 23

¿Cual es el tipo de datos de "value package", visits y user?

```
TYPE "value package"  
string  
TYPE user  
none  
TYPE visits
```

Sección 3

Manejo de claves

Inciso 24

Obtener todas las claves que empiecen con "v".

```
KEYS v*
```

Preguntar si es con keys o con el comando SCAN

Inciso 25

Obtener todas las claves que contengan la letra "t".

```
KEYS *t*
```

Inciso 26

Obtener todas las claves que terminan con "age".

KEYS *age

Inciso 27

Renombrar la clave "package" por "bariloche package".

```
RENAME "value package" "bariloche package"
```

Puse value package porque no existia la key package, en caso de que no sea un error de tipeo, sería RENAMENX, para que valide que exista antes de intentar renombrarla.

Inciso 28

¿Qué comando se utiliza para renombrar una clave solo si el nombre destino no existe aún?

RENAMENX se utiliza para ello.

Inciso 29

Eliminar todas las claves.

```
FLUSHDB
```

Sección 4

Expiración de claves

Inciso 30

Agregar una clave agency con el valor "Cronos Tours".

```
SET agency "Cronos Tours"
```

Inciso 31

¿Cuál es el tiempo de vida (TTL) de la clave agency?

```
EXPIRETIME agency
```

```
TTL agency
```

Es -1 porque su tiempo de vida es "indefinido", ya que no va a expirar por si sola.

No se cual de los dos comandos es el adecuado

Inciso 32

Agregar una expiración de 30 segundos a la clave agency.

```
EXPIRE agency 30
```

Inciso 33

¿Cuál es el tiempo de vida de la clave agency luego de agregar la expiración?

El tiempo de vida va bajando por segundo, de 29 a 28 y así hasta llegar a 0.

Inciso 34

Pasados los 30 segundos: ¿cuál es el TTL de agency? ¿Que retorna si se solicita el valor de agency?

Una vez que termina ese tiempo pasa a tener el valor -2, indicando que la clave ya expiró y fue eliminada.

Inciso 35

Agregar una clave agency con el valor "Cronos Tours" que expire en 20 segundos desde su creación.

Sección 5

Listas

Inciso 36

Insertar una lista llamada pets con el valor "dog".

```
LPUSH pets "dog"
```

Inciso 37

¿Qué sucede si se ejecuta el comando GET sobre pets? ¿Cómo se obtienen los valores de una lista?

Cuando ejecutas `GET`, Redis espera ir a buscar un único valor directo. Si intentas aplicar `GET` sobre una lista, el sistema se frena y te devuelve el error `WRONGTYPE Operation against a key holding the wrong kind of value`.

Los valores de una lista se pueden obtener de diferentes formas, como las siguientes:

1. `LRANGE` (Leer un rango de elementos)
 1. Es el comando estándar para consultar el contenido sin modificar la lista. Requiere un índice de inicio y uno de fin (basados en cero).
2. `LINDEX` (Obtener un elemento por su posición)
 1. Sirve para buscar y devolver un único elemento que se encuentra en una posición (índice) específica.
3. `LPOP` / `RPOP` (Extraer y remover elementos)
 1. Estos comandos **modifican la lista**: obtienen el elemento del extremo seleccionado y lo eliminan definitivamente de la estructura.
 2. `LPOP` remueve el primer elemento

- 3. RPOP el último elemento
- 4. **LLEN** (Obtener el tamaño)
 - 1. Nos da el tamaño de la lista

Inciso 38

Agregar a la lista pets el valor "cat" por la izquierda.

```
LPUSH pets "cat"
```

Inciso 39

Agregar a la lista pets el valor "fish" por la derecha.

```
R PUSH pets "fish"
```

Inciso 40

¿Qué tipo de dato es el valor de pets?

```
TYPE pets
```

Pets es de tipo lista

Inciso 41

Eliminar el valor del extremo izquierdo de la lista.

```
LPOP pets
```

Inciso 42

Eliminar el valor del extremo derecho de la lista.

Inciso 43

Agregar a una clave "vuelo:ar389" los valores: aep, mdz, brc, nqn y mdq.

```
LPUSH "vuelo:ar389" "aep" "mdz" "brc" "nqn" "mdq"
```

Inciso 44

Ordenar los valores de la lista "vuelo:ar389". ¿Qué sucede si se solicitan todos los valores de la lista luego de ordenarla?

```
SORT "vuelo:ar389" ALPHA
```

El comando `SORT` es de "solo lectura". Lo que hace Redis es ir a la memoria, duplicar los elementos en caliente, ordenarlos en una estructura temporal y **mostrártelos ordenados en la pantalla, pero sin tocar la lista original**.

Si inmediatamente después de usar `SORT` ejecutás un `LRANGE "vuelo:ar389" 0 -1`, vas a ver los elementos **en el mismo orden exacto en el que fueron insertados originalmente** (con `LPUSH` o `RPUSH`).

Si el objetivo real es ordenar la lista y que ese resultado quede guardado en la base de datos de forma permanente, tenés que usar el argumento `STORE`.

Este argumento agarra el resultado del ordenamiento y lo mete dentro de una clave nueva

Inciso 45

Insertar el valor "fte" inmediatamente después de "brc".

```
LINSERT "vuelo:ar389" AFTER "brc" "fte"
```

Inciso 46

Insertar el valor "ush" inmediatamente antes de "fte".

```
LINSERT "vuelo:ar389" BEFORE "fte" "ush"
```

Inciso 47

Modificar el último elemento de la lista por "sla".

```
LSET "vuelo:ar389" -1 "sla"
```

Consideraciones:

La clave y la posición TIENEN que existir: A diferencia de `LPUSH` o `RPUSH` (que crean la lista si no existe), `LSET` **solo sirve para sobrescribir**. Si la lista está vacía o si intentás usar un índice que está fuera de rango (por ejemplo, poner índice `5` en una lista que solo tiene 2 elementos), Redis te va a tirar error.

Inciso 48

Obtener la cantidad de elementos de "vuelo:ar389".

```
LLEN "vuelo:ar389"
```

Inciso 49

Obtener el tercer valor de "vuelo:ar389".

```
LINDEX "vuelo:ar389" 2
```

Inciso 50

Eliminar el valor "aep" de "vuelo:ar389".

```
LREM "vuelo:ar389" 0 "aep"
```

Inciso 51

Quedarse únicamente con los valores de las posiciones 3 a 5 de "vuelo:ar389".

```
LTRIM "vuelo:ar389" 2 4
```

Inciso 52

Agregar en "vuelo:ar389" el valor "fte". ¿Cuántas veces aparece ahora en la lista?

Dos veces aparece, al principio y al final de la lista

Sección 6

Conjuntos - Sets

Inciso 53

Agregar un conjunto llamado airports con los siguientes valores:

eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juj aep tuc eqs.

```
SADD "airports" "eze" "aep" "nqn" "mz" "mdq" "ush"  
"fte" "sla" "aep" "nqn" "brc" "cpc" "juj" "aep"  
"tuc" "eqs"
```

Inciso 54

¿Cuántos valores tiene el conjunto? ¿Por qué puede diferir de la cantidad de valores ingresados?

Respuesta:

El conjunto tiene exactamente **13 valores**.

Difiere de la cantidad de valores ingresados (que fueron 16 en total) debido a la naturaleza y las propiedades fundamentales del tipo de dato **Set (Conjunto)** en Redis:

1. **Unicidad de los elementos:** Los Sets en Redis son colecciones de cadenas **únicas y no repetidas**.
2. **Filtrado automático:** Cuando intentás insertar valores duplicados en un solo comando **SADD**, Redis los procesa uno por uno. Si el elemento ya existe dentro del conjunto, el motor lo ignora silenciosamente y no lo vuelve a agregar.

Inciso 55

Listar los valores del conjunto airports.

```
SMEMBERS airports
```

Inciso 56

Quitar el valor "cpc" del conjunto airports.

```
SREM airports "cpc"
```

Inciso 57

Quitar un valor aleatorio del conjunto airports.

```
SPOP airports
```

Inciso 58

¿Qué cantidad de valores tiene airports ahora?

Respuesta:

Tiene la cantidad de 11 elementos

Inciso 59

Comprobar si "cpc" es miembro del conjunto airports.

```
SISMEMBER airports "cpc"
```

La respuesta binaria en formato de entero:

- **(integer) 1**: Significa **Verdadero**. El elemento **"cpc"** sí existe dentro del conjunto **airports**.
- **(integer) 0**: Significa **Falso**. El elemento no existe en el conjunto (o directamente la clave que pasaste no existe en la base de datos).

Inciso 60

Mover los valores "sla" y "juj" a un nuevo conjunto denominado noa_airports.

```
SMOVE airports noa_airports "sla"  
SMOVE airports noa_airports "juj"
```

Inciso 61

Obtener la unión de los conjuntos airports y noa_airports.
¿Modifica los conjuntos originales?

```
SUNION airports noa_airports
```


Cuando ejecutamos `SUNION`, Redis va a la memoria, lee los elementos de ambos conjuntos, realiza la unión matemática en caliente (eliminando cualquier duplicado que pueda haber entre ambos) y **te muestra el resultado en la pantalla**. Los conjuntos originales (`airports` y `noa_airports`) quedan exactamente iguales a como estaban antes de correr el comando.

Para persistir se usa `SUNIONSTORE`.

Inciso 62

Realizar la unión de `airports` y `noa_airports` y almacenar el resultado en un nuevo conjunto llamado `total_airports`.

```
SUNIONSTORE total_airports airports noa_airports
```

Inciso 63

Realizar la intersección entre `total_airports` y `noa_airports`.

```
SINTER total_airports noa_airports
```

Inciso 64

Realizar la diferencia entre `total_airports` y `noa_airports`.

```
SDIFF total_airports noa_airports
```

Sección 7

Conjuntos Ordenados (Sorted Sets)

Inciso 65

Agregar a un conjunto ordenado llamado `passengers` los siguientes datos (score nombre):

2.5 federico 4 alejandra 3 julian 1 ivan 2 tomas 2 luciana 2.4 natalia

```
ZADD passengers 2.5 "federico" 4 "alejandra" 3  
"julian" 1 "ivan" 2 "tomas" 2 "luciana" 2.4  
"natalia"
```

Inciso 66

Obtener los valores del conjunto passengers.

```
ZRANGE passengers 0 -1 # Mostrar las claves en orden  
ascendente  
ZRANGE passengers 0 -1 WITHSCORES # Para mostrar el  
valor asociado
```

Inciso 67

Actualizar el score de luciana a 2.7.

```
ZADD passengers 2.7 "luciana"
```

Inciso 68

Agregar al conjunto passengers a silvia con score 5.1.

```
ZADD passengers 5.1 "silvia"
```

Inciso 69

Incrementar en 2 el score de alejandra.

```
ZINCRBY passengers 2 "alejandra"
```

Inciso 70

Obtener los valores del conjunto passengers con sus scores.

```
ZRANGE passengers 0 -1 WITHSCORES
```

Inciso 71

Obtener los valores del conjunto passengers con sus scores en orden inverso.

```
ZRANGE passengers 0 -1 WITHSCORES REV
```

Inciso 72

Obtener la cantidad de elementos del conjunto passengers.

```
ZCARD passengers
```

Inciso 73

Obtener la cantidad de elementos que tienen scores entre 2 y 3.

```
ZCOUNT passengers 2 3
```

Inciso 74

Obtener el ranking de julian en el conjunto passengers.

```
ZRANK passengers "julian"
```

Inciso 75

Obtener el score de tomas en el conjunto passengers.

```
ZSCORE passengers "tomas"
```

Inciso 76

Extraer el elemento con menor score del conjunto passengers.

```
ZPOPMIN passengers
```

Inciso 77

Extraer el elemento con mayor score del conjunto passengers.

```
ZPOPMAX passengers
```

Inciso 78

Eliminar del conjunto passengers al valor silvia.

```
ZREM passengers "silvia"
```